

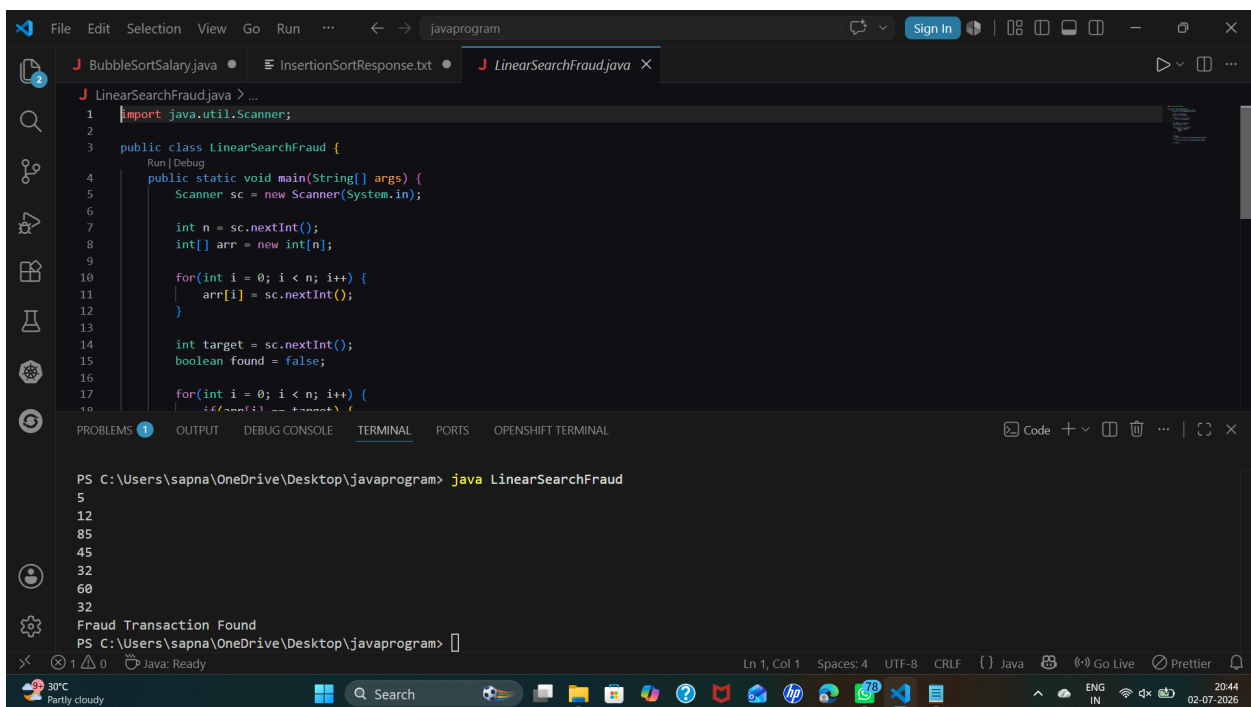
MODULE:-2

Name:-kumari sapna

Course:-B.Tech

Branch:-AIML

Q1. Fraud Transaction Detection (Linear Search) Problem: A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists. Write a program using Linear Search



The screenshot shows an IDE with a Java file named `LinearSearchFraud.java`. The code implements a linear search algorithm to find a target transaction ID in an array. The terminal output shows the program running successfully and finding the target.

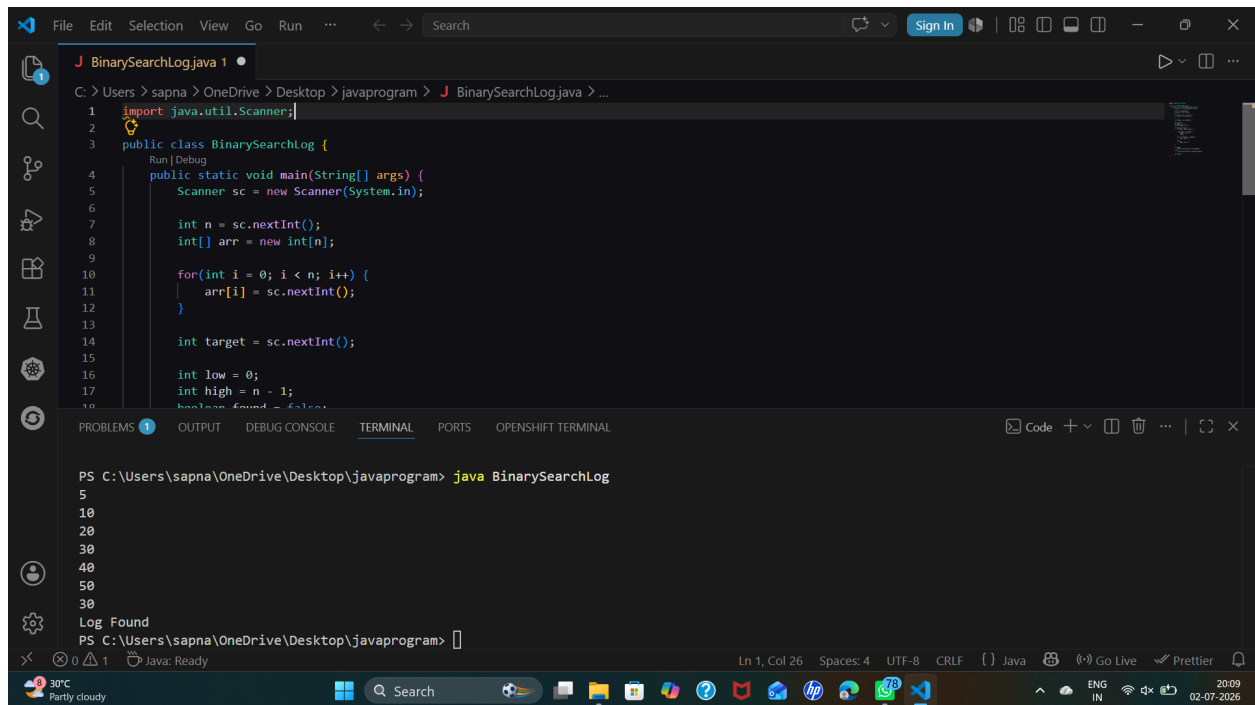
```
1 import java.util.Scanner;
2
3 public class LinearSearchFraud {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9
10        for(int i = 0; i < n; i++) {
11            arr[i] = sc.nextInt();
12        }
13
14        int target = sc.nextInt();
15        boolean found = false;
16
17        for(int i = 0; i < n; i++) {
18            if(arr[i] == target) {
19                found = true;
20            }
21        }
22
23        if(found) {
24            System.out.println("Fraud Transaction Found");
25        } else {
26            System.out.println("Fraud Transaction Not Found");
27        }
28    }
29 }
```

Terminal Output:

```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java LinearSearchFraud
5
12
85
45
32
60
32
Fraud Transaction Found
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

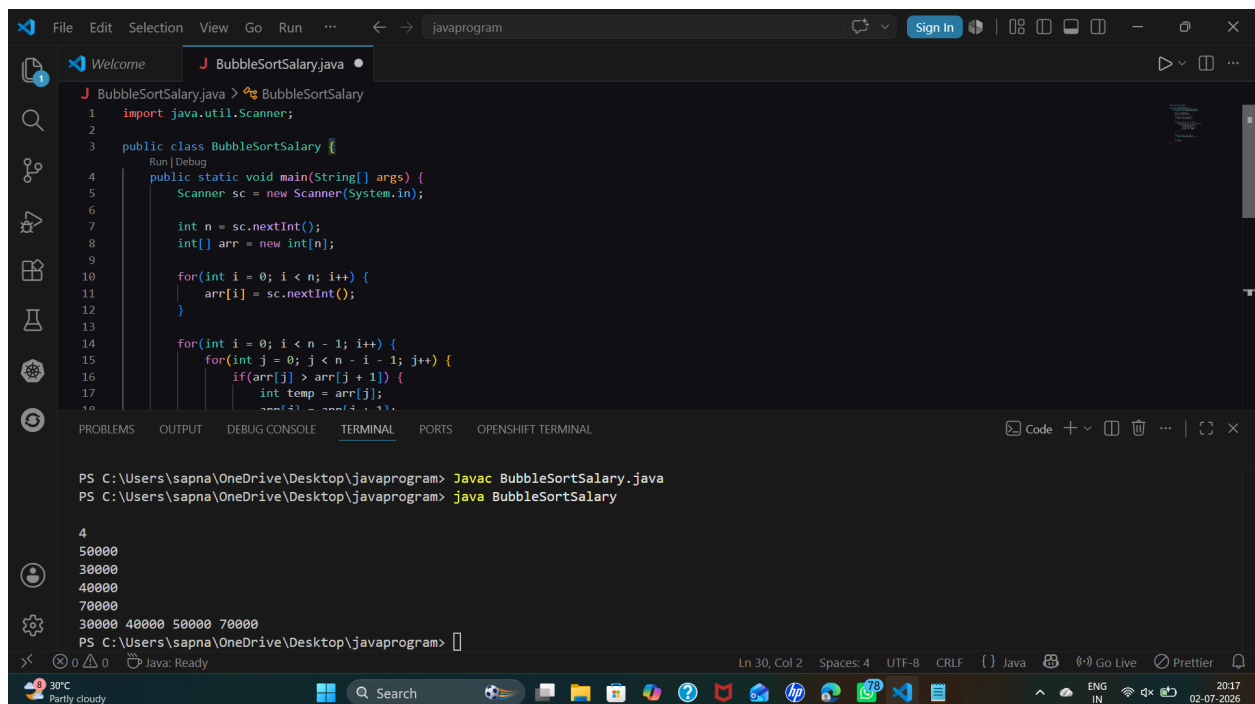
Q2. Server Log Search (Binary Search) Problem: A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists. Write a program using Binary

Search.



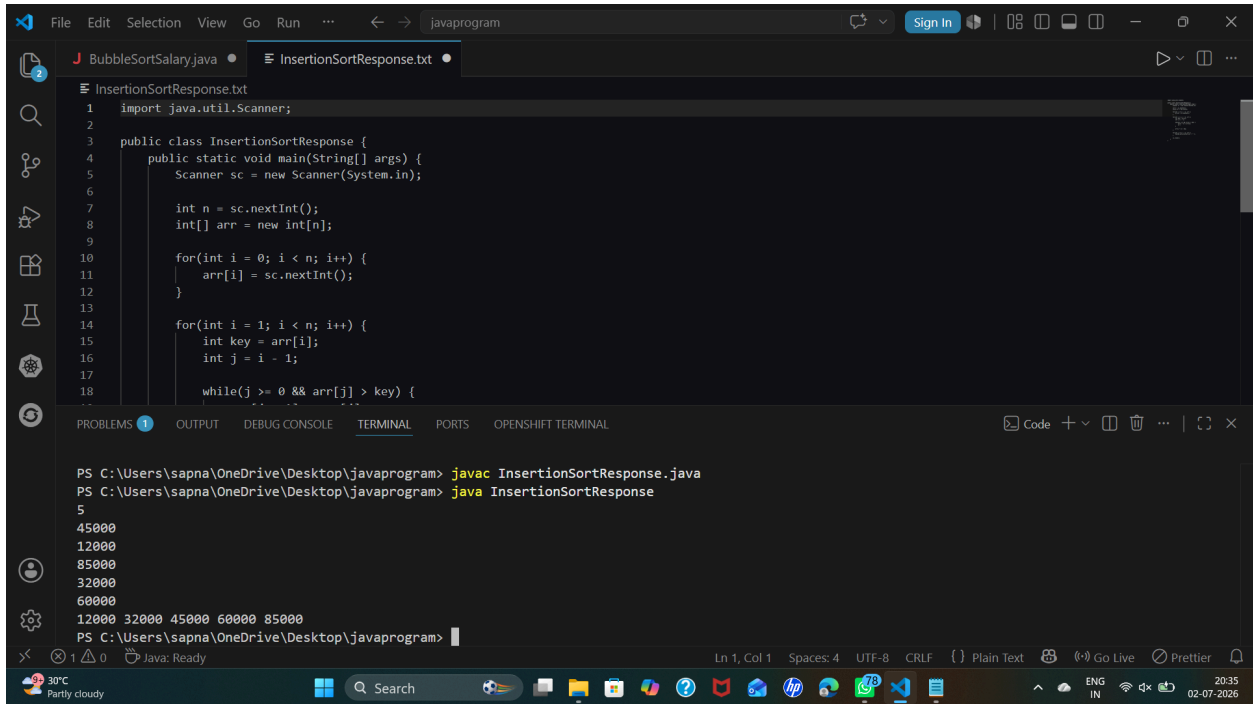
```
File Edit Selection View Go Run ... Search
J BinarySearchLog.java 1
C:\Users\sapna> OneDrive > Desktop > javaprogram > J BinarySearchLog.java > ...
1 import java.util.Scanner;
2
3 public class BinarySearchLog {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9
10        for(int i = 0; i < n; i++) {
11            arr[i] = sc.nextInt();
12        }
13
14        int target = sc.nextInt();
15
16        int low = 0;
17        int high = n - 1;
18        boolean found = false;
19
20        while (low <= high) {
21            int mid = (low + high) / 2;
22            if (arr[mid] == target) {
23                found = true;
24                break;
25            }
26            else if (arr[mid] < target) {
27                low = mid + 1;
28            }
29            else {
30                high = mid - 1;
31            }
32        }
33
34        if (found) {
35            System.out.println("Target found at index: " + mid);
36        }
37        else {
38            System.out.println("Target not found");
39        }
40    }
41}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS OPENSIFT TERMINAL
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java BinarySearchLog
5
10
20
30
40
50
30
Log Found
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q3 Sort Employee Salaries (Bubble Sort) Problem Statement: A company wants to sort employee salaries in ascending order for payroll analysis. Write a program using Bubble Sort .



```
File Edit Selection View Go Run ... javaprogram
Welcome J BubbleSortSalary.java
J BubbleSortSalary.java > BubbleSortSalary
1 import java.util.Scanner;
2
3 public class BubbleSortSalary {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9
10        for(int i = 0; i < n; i++) {
11            arr[i] = sc.nextInt();
12        }
13
14        for(int i = 0; i < n - 1; i++) {
15            for(int j = 0; j < n - i - 1; j++) {
16                if(arr[j] > arr[j + 1]) {
17                    int temp = arr[j];
18                    arr[j] = arr[j + 1];
19                    arr[j + 1] = temp;
20                }
21            }
22        }
23
24        for(int i = 0; i < n; i++) {
25            System.out.print(arr[i] + " ");
26        }
27        System.out.println();
28    }
29}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS OPENSIFT TERMINAL
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> javac BubbleSortSalary.java
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java BubbleSortSalary
4
50000
30000
40000
70000
30000 40000 50000 70000
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q4. Sort Website Response Time (Insertion Sort) Problem Statement: A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position. Write a program using Insertion Sort



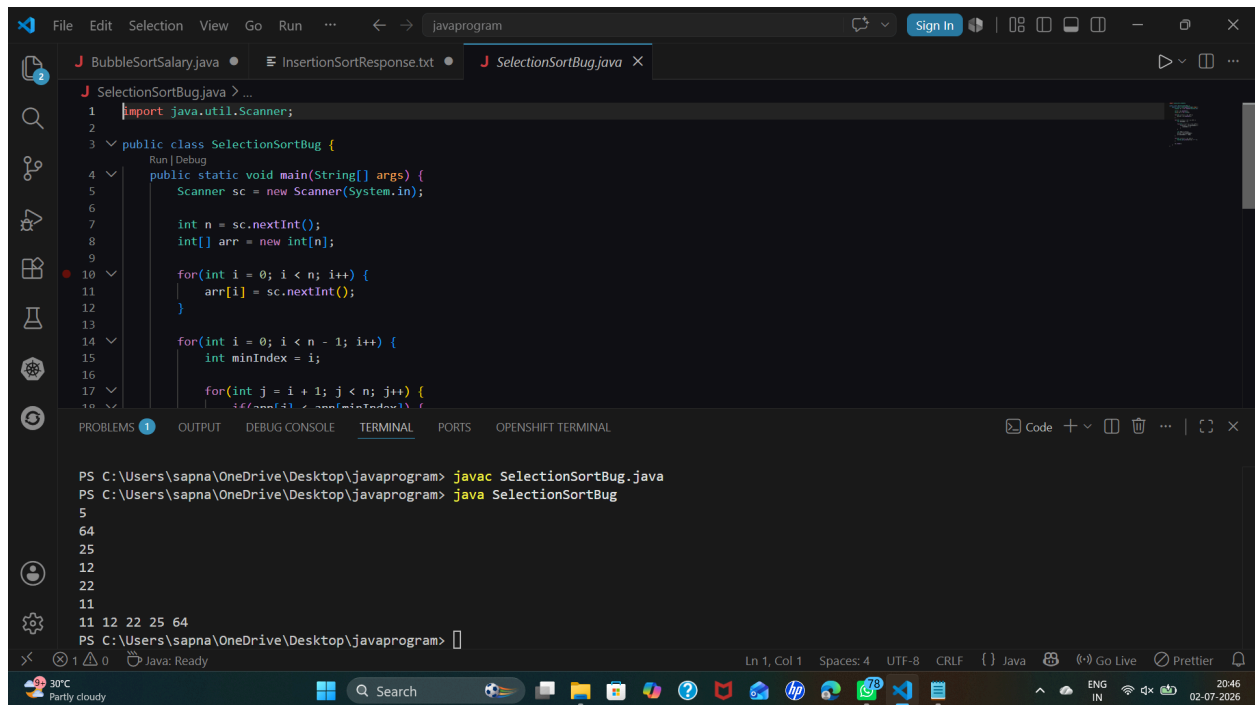
The screenshot shows an IDE with a Java file named `InsertionSortResponse.txt`. The code implements an Insertion Sort algorithm. The `main` method reads integers from `System.in` and sorts them. The output in the terminal shows the input sequence: 5, 45000, 12000, 85000, 32000, 60000, and the sorted sequence: 12000, 32000, 45000, 60000, 85000.

```
1 import java.util.Scanner;
2
3 public class InsertionSortResponse {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9
10        for(int i = 0; i < n; i++) {
11            arr[i] = sc.nextInt();
12        }
13
14        for(int i = 1; i < n; i++) {
15            int key = arr[i];
16            int j = i - 1;
17
18            while(j >= 0 && arr[j] > key) {
19                ...
20            }
21        }
22    }
23 }
```

```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> javac InsertionSortResponse.java
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java InsertionSortResponse
5
45000
12000
85000
32000
60000
12000 32000 45000 60000 85000
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q5. Prioritize Bug Reports (Selection Sort) Problem Statement: A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending

order. Write a program using Selection Sort.



The screenshot shows an IDE with a Java file named `SelectionSortBug.java`. The code implements a Selection Sort algorithm. The `main` method reads an integer `n` and an array `arr` of size `n` from the user. It then performs the selection sort by finding the minimum element in the unsorted part of the array and swapping it with the first element of the unsorted part.

```
1 import java.util.Scanner;
2
3 public class SelectionSortBug {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9
10        for(int i = 0; i < n; i++) {
11            arr[i] = sc.nextInt();
12        }
13
14        for(int i = 0; i < n - 1; i++) {
15            int minIndex = i;
16
17            for(int j = i + 1; j < n; j++) {
18                if(arr[j] < arr[minIndex]) {
19                    minIndex = j;
20                }
21            }
22            // Swap arr[i] and arr[minIndex]
23            int temp = arr[i];
24            arr[i] = arr[minIndex];
25            arr[minIndex] = temp;
26        }
27
28        for(int i = 0; i < n; i++) {
29            System.out.print(arr[i] + " ");
30        }
31        System.out.println();
32    }
33 }
```

The terminal output shows the program being compiled and run. The user inputs `5` for `n` and `64 25 12 22 11` for the array elements. The sorted array is printed as `11 12 22 25 64`.

```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> javac SelectionSortBug.java
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java SelectionSortBug
5
64
25
12
22
11
11 12 22 25 64
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```